

**IDENTIFIKASI PROVINSI DENGAN PERFORMA TERBAIK
PADA OSN MENGGUNAKAN METODEODOLOGI CRISP-DM
DAN MACHINE LEARNING**



DISUSUN OLEH :

NAMA : RAMA BRAMANTHIYO SUSANTO PUTRA

NIM : 105841115123

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2025

BAB I

PENDAHULUAN

A. Latar Belakang

Olimpiade Sains Nasional (OSN) merupakan indikator utama dalam memetakan kualitas pendidikan sains tingkat sekolah menengah di Indonesia. Selama ini, evaluasi keberhasilan suatu provinsi dalam ajang ini sering kali hanya didasarkan pada "total kuantitas medali" yang diraih. Metode evaluasi konvensional ini cenderung bias karena secara alami menguntungkan provinsi dengan populasi besar (seperti di Pulau Jawa) yang mampu mengirimkan delegasi peserta dalam jumlah masif.

Akibatnya, provinsi-provinsi kecil yang mungkin mengirimkan sedikit peserta namun memiliki tingkat keberhasilan (efisiensi) yang tinggi sering kali luput dari perhatian. Diperlukan sebuah pendekatan berbasis data (*Data Science*) untuk mengubah cara pandang evaluasi dari sekadar "Jumlah Medali" menjadi "Efisiensi Kualitas" atau *Win Rate*. Dengan memanfaatkan metodologi CRISP-DM dan algoritma *Machine Learning* K-Means, penelitian ini bertujuan untuk mengelompokkan provinsi berdasarkan karakteristik efisiensi mereka, sehingga didapatkan peta kualitas pendidikan yang lebih adil dan objektif.

B. Rumusan Masalah

1. Bagaimana cara mengukur performa provinsi secara adil dengan meminimalisir bias jumlah populasi peserta?
2. Bagaimana pola pengelompokan (clustering) provinsi di Indonesia jika ditinjau dari aspek efisiensi prestasi?
3. Bagaimana membangun sistem visualisasi interaktif yang memudahkan pemangku kepentingan memahami data kompleks tersebut?

C. Batasan Masalah

1. Dataset yang digunakan mencakup rentang tahun 2009 hingga 2025.
2. Data tahun 2009–2024 memiliki keterbatasan label medali (hanya tercatat "Finalis"), sehingga status "Finalis" dianggap sebagai poin prestasi (skor 1) dalam penelitian ini.
3. Algoritma yang digunakan terbatas pada K-Means Clustering.

D. Tujuan Penelitian

1. Menerapkan siklus CRISP-DM secara utuh dalam penyelesaian masalah data riil.
2. Mengidentifikasi provinsi-provinsi yang masuk dalam kategori "Sangat Efisien" (High Performance) meskipun jumlah pesertanya sedikit.
3. Menghasilkan web dashboard berbasis Gradio yang dapat diakses publik.

BAB II

LANDASAN TEORI

A. CRISP-DM

Data Science adalah bidang interdisipliner yang menggunakan metode ilmiah, proses, algoritma, dan sistem untuk mengekstrak pengetahuan dan wawasan dari data dalam berbagai bentuk, baik terstruktur maupun tidak terstruktur.

Dalam penerapannya, standar industri yang paling banyak diakui adalah CRISP-DM (Cross-Industry Standard Process for Data Mining).

Metodologi CRISP-DM terdiri dari enam tahapan siklus hidup yang bersifat iteratif (berulang), yaitu:

1. Business Understanding: Memahami tujuan proyek dari sudut pandang bisnis atau institusi.
2. Data Understanding: Mengumpulkan, mendeskripsikan, dan mengeksplorasi data awal.
3. Data Preparation: Tahap pembersihan, transformasi, dan konstruksi data agar siap dimodelkan.
4. Modeling: Pemilihan dan penerapan teknik pemodelan (algoritma).
5. Evaluation: Mengevaluasi model untuk memastikan kesesuaian dengan tujuan bisnis awal.
6. Deployment: Penerapan hasil analisis ke dalam operasional sehari-hari.

B. K-Means Clustering

Clustering atau pengelompokan adalah teknik Data Mining yang bersifat Unsupervised Learning (pembelajaran tanpa supervisi), di mana data dikelompokkan tanpa adanya label target yang ditentukan sebelumnya. Salah satu algoritma yang paling populer dan efisien adalah K-Means.

Prinsip kerja K-Means adalah mempartisi data yang ada ke dalam kelompok (klaster). Algoritma ini bekerja secara iteratif dengan menetapkan pusat klaster (centroid) secara acak, kemudian mengelompokkan setiap titik data ke centroid terdekat berdasarkan jarak Euclidean. Pusat klaster kemudian diperbarui dengan menghitung rata-rata dari anggota klaster, dan proses ini

berulang hingga posisi centroid stabil (konvergen). Algoritma ini dipilih dalam penelitian ini karena kemampuannya yang baik dalam menangani data numerik dan kemudahannya untuk diinterpretasikan dalam konteks segmentasi wilayah.

C. Evaluasi Cluster

Karena clustering tidak memiliki "kunci jawaban" (label benar/salah), evaluasi dilakukan menggunakan metrik internal:

1. **Elbow Method:** Sebuah teknik heuristik visual untuk menentukan jumlah kluster yang optimal. Metode ini memplot nilai inersia (Sum of Squared Errors) terhadap jumlah kluster. Titik di mana penurunan inersia mulai melandai secara drastis (membentuk siku) dianggap sebagai jumlah kluster terbaik.
2. **Silhouette Score:** Metode validasi yang mengukur seberapa mirip suatu objek dengan klasternya sendiri (cohesion) dibandingkan dengan kluster lain (separation). Nilai koefisien berkisar antara -1 hingga +1. Nilai yang mendekati +1 menunjukkan bahwa objek terkelompok dengan sangat baik dan terpisah jauh dari kluster tetangga.

D. Gradio

Gradio adalah pustaka open-source berbasis Python yang dirancang khusus untuk mendemonstrasikan model Machine Learning secara cepat. Gradio memungkinkan pengembang untuk membuat antarmuka web (web interface) yang interaktif hanya dengan beberapa baris kode. Fitur unggulannya meliputi komponen input yang beragam (slider, dropdown, upload file) dan kemampuan untuk membuat tautan publik (public sharing link), sehingga aplikasi yang berjalan di server lokal dapat diakses oleh pengguna lain melalui internet tanpa perlu konfigurasi server yang rumit.

BAB III

METODOLOGI PENELITIAN

A. Business Understanding

Tahap pertama berfokus pada pendefinisian masalah dari perspektif pemangku kepentingan (stakeholders), dalam hal ini adalah pengamat pendidikan atau Dinas Pendidikan. Masalah utamanya adalah bias evaluasi berbasis kuantitas. Oleh karena itu, tujuan analitik proyek ini adalah mengubah data transaksional peserta menjadi wawasan strategis berupa segmentasi provinsi. Kriteria kesuksesan proyek ini adalah jika model mampu menghasilkan kelompok provinsi dengan karakteristik efisiensi yang unik dan dapat dijelaskan secara logis (interpretable).

B. Data Understanding

Data yang digunakan dalam penelitian ini bersumber dari DATASET OSN.csv. Dataset ini berisi rekam jejak kepesertaan OSN tingkat SMP. Atribut data meliputi:

1. Nama Peserta: Identitas siswa.
2. Sekolah dan Provinsi: Asal daerah peserta.
3. Medali: Status pencapaian (Emas, Perak, Perunggu, Finalis).
4. Tahun: Tahun pelaksanaan lomba (2009–2025).

Pada tahap eksplorasi awal, ditemukan tantangan kualitas data. Data pada rentang tahun 2009 hingga 2024 tidak memiliki pencatatan medali yang konsisten (mayoritas hanya tercatat sebagai "Finalis"), sedangkan pencatatan medali lengkap (Emas/Perak/Perunggu) baru tersedia secara konsisten pada tahun 2025. Fakta ini menjadi dasar pertimbangan strategi pada tahap persiapan data.

C. Data Preparation

Untuk mengatasi perbedaan data tersebut, dilakukan proses pembersihan dan pembobotan nilai prestasi agar data lama tetap bisa diperhitungkan setara dengan data baru. Data yang awalnya berlevel individu siswa kemudian diubah

(agregasi) menjadi level provinsi untuk mendapatkan dua variabel kunci: total jumlah peserta dan rasio kemenangan (Win Rate). Sebelum masuk ke pemodelan, data dinormalisasi agar perbedaan skala angka antara jumlah peserta yang ratusan dan persentase kemenangan tidak membuat hasil analisis menjadi bias.

D. Modeling Strategy

Pada tahap ini, algoritma *K-Means Clustering* digunakan untuk mengelompokkan provinsi berdasarkan karakteristik kemiripan data yang telah diolah. Model dirancang secara fleksibel menggunakan pustaka Scikit-Learn, memungkinkan pengguna untuk bereksperimen dengan mengubah jumlah kelompok (klaster) sesuai kebutuhan analisis guna menemukan pola tersembunyi yang paling relevan.

E. Evaluation

Mengingat metode ini tidak memiliki "kunci jawaban" benar atau salah, validasi dilakukan menggunakan pendekatan statistik internal seperti Elbow Method dan Silhouette Score. Evaluasi ini bertujuan memastikan bahwa jumlah kelompok yang dipilih adalah yang paling efisien dan pemisahan antar-kelompok provinsi benar-benar tegas serta valid, bukan sekadar pengelompokan yang terjadi karena kebetulan

F. Deployment

Sebagai langkah akhir, hasil analisis yang kompleks diterjemahkan ke dalam bentuk Dashboard Interaktif berbasis web menggunakan *Gradio*. Alat ini memungkinkan pengguna awam untuk melihat visualisasi posisi provinsi dalam grafik dua dimensi (Kuantitas vs Kualitas) dan melakukan analisis perbandingan secara mudah melalui peramban web tanpa perlu melakukan instalasi teknis yang rumit.

BAB IV

IMPLEMENTASI DAN PEMBAHASAN

A. Analisis Pra-Pemrosesan Data

Setelah proses agregasi, didapatkan tabel data provinsi yang bersih dari nilai null. Isu data tahun 2009-2024 yang hanya berisi "Finalis" berhasil diatasi dengan memberikan bobot nilai 1, sehingga provinsi yang aktif di tahun-tahun tersebut tetap terdeteksi partisipasinya dalam analisis tren jangka panjang.

B. Penjelasan Teknis Kode Program

Implementasi sistem dilakukan secara modular dalam beberapa blok kode (cell) menggunakan Jupyter Notebook. Berikut adalah uraian mendalam mengenai fungsi dan logika teknis dari setiap blok kode :

1. Import Pustaka dan Konfigurasi Awal

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import gradio as gr
import warnings

# Mengabaikan warning agar output lebih bersih
warnings.filterwarnings('ignore')
```

Pemanggilan (*import*) fungsi-fungsi spesifik ke dalam memori. Pustaka pandas dan numpy dipanggil untuk menangani operasi matriks tabel. Pustaka sklearn dipanggil untuk mengakses algoritma KMeans, StandardScaler (normalisasi), dan silhouette_score (validasi). Sel ini juga mengatur konfigurasi sistem, seperti menonaktifkan pesan peringatan (*warnings*) agar *output* program terlihat bersih saat presentasi.

2. Pra-pemrosesan Data (Data Loading & Cleaning)

```
# --- FUNGSI LOAD DATA ---
def load_and_clean_data(file_path):
    try:
        # Coba separator titik koma (;)
        df = pd.read_csv(file_path, sep=';', encoding='latin-1')
    except:
        # Fallback separator koma (,)
        df = pd.read_csv(file_path, sep=',', encoding='latin-1')

    # Bersihkan nama kolom
    df.columns = [c.strip() for c in df.columns]

    # Isi Medali kosong
    df['Medali'] = df['Medali'].fillna('Non-Medalis')

    # Feature Engineering: Identifikasi Juara
    medali_valid = ['Emas', 'Perak', 'Perunggu', 'Gold', 'Silver', 'Bronze']
    df['Is_Juara'] = df['Medali'].apply(lambda x: 1 if any(m in str(x) for m in medali_valid) else 0)

    # Feature Engineering: Skor Kualitas
    def hitung_bobot(medali):
        m = str(medali).lower()
        if 'emas' in m: return 4
        elif 'perak' in m: return 3
        elif 'perunggu' in m: return 2
        else: return 1

    df['Skor_Kualitas'] = df['Medali'].apply(hitung_bobot)

    # Pastikan tahun integer
    df = df.dropna(subset=['Tahun'])
    df['Tahun'] = df['Tahun'].astype(int)

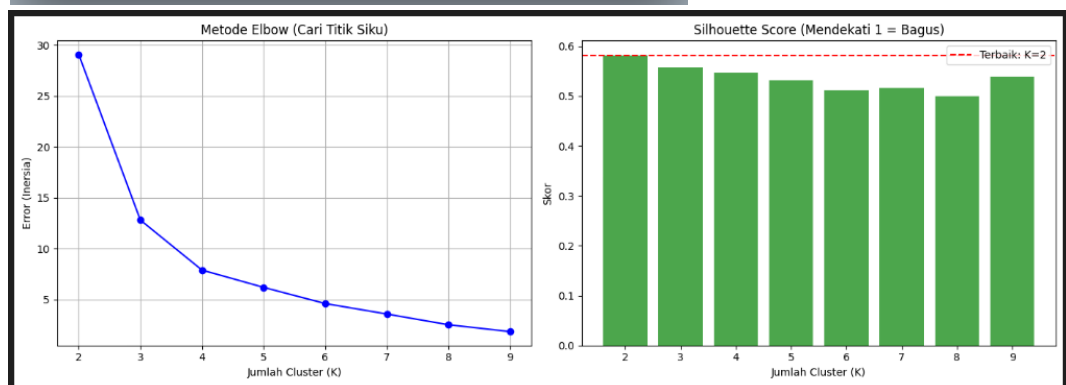
    return df

# --- EKSEKUSI ---
filename = 'DATASET_OSN.csv' # Pastikan nama file sesuai
try:
    df_raw = load_and_clean_data(filename)
    print(f"✅ Data Terload: {df_raw.shape[0]} baris.")
    print(df_raw.head(3))
except:
    print("❌ File tidak ditemukan. Upload 'DATASET_OSN.csv' dulu.")
    df_raw = pd.DataFrame()
```

Fungsi `load_and_clean_data` dirancang dengan mekanisme error handling (try-except) untuk mendeteksi otomatis apakah file CSV menggunakan pemisah titik koma atau koma. Inovasi terpenting di sini adalah Logika Pembobotan Prestasi. Mengingat data tahun 2009–2024 dominan hanya mencatat status "Finalis", program mengonversi status teks menjadi nilai numerik: Emas (4), Perak (3), Perunggu (2), dan Finalis (1). Logika ini "menyelamatkan" ribuan data historis agar tetap memiliki nilai partisipasi dan tidak dianggap nol oleh algoritma.

3. Evaluasi Model (Elbow & Silhouette Analysis)

```
1 # --- ANALISIS EVALUASI CLUSTER ---
2 def evaluasi_cluster(df, tahun="Semua Tahun"):
3     if df.empty: return
4
5     # 1. Filter Data
6     if tahun != "Semua Tahun":
7         df_filter = df[df['Tahun'] == int(tahun)]
8     else:
9         df_filter = df.copy()
10
11     # 2. Agregasi Data (Siswa -> Provinsi)
12     df_agg = df_filter.groupby('Provinsi').agg({
13         'Nama Peserta': 'count',
14         'Is_Juara': 'sum'
15     }).reset_index()
16
17     # PENTING: Rename kolom agar tidak KeyError
18     df_agg.rename(columns={'Nama Peserta': 'Total_Peserta', 'Is_Juara': 'Total_Medali'}, inplace=True)
19
20     # Hitung Win Rate
21     df_agg['Win_Rate'] = (df_agg['Total_Medali'] / df_agg['Total_Peserta']) * 100
22     df_agg = df_agg[df_agg['Total_Peserta'] > 0]
23
24     if len(df_agg) < 5:
25         print("Data terlalu sedikit untuk evaluasi grafik.")
26         return
27
28     # 3. Scaling
29     X = df_agg[['Total_Peserta', 'Win_Rate']]
30     scaler = StandardScaler()
31     X_scaled = scaler.fit_transform(X)
32
33     # 4. Loop K-Means (Mencari K Terbaik)
34     inertias = []
35     sil_scores = []
36     k_range = range(2, min(10, len(df_agg)))
37
38     for k in k_range:
39         km = KMeans(n_clusters=k, random_state=42, n_init=10)
40         km.fit(X_scaled)
41         inertias.append(km.inertia_)
42         sil_scores.append(silhouette_score(X_scaled, km.labels_))
43
44     # 5. Plotting
45     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
46
47     # Grafik Elbow
48     ax1.plot(k_range, inertias, 'bo-')
49     ax1.set_title("Metode Elbow (Cari Titik Siku)")
50     ax1.set_xlabel("Jumlah Cluster (K)")
51     ax1.set_ylabel("Error (Inersia)")
52     ax1.grid(True)
53
54     # Grafik Silhouette
55     ax2.bar(k_range, sil_scores, color='green', alpha=0.7)
56     ax2.set_title("Silhouette Score (Mendekati 1 = Bagus)")
57     ax2.set_xlabel("Jumlah Cluster (K)")
58     ax2.set_ylabel("Skor")
59
60     # Tanda Skor Tertinggi
61     best_score = max(sil_scores)
62     best_k = k_range[sil_scores.index(best_score)]
63     ax2.axhline(best_score, color='red', linestyle='--', label=f'Terbaik: K={best_k}')
64     ax2.legend()
65
66     plt.tight_layout()
67     plt.show()
68     print(f"👉 Rekomendasi: Gunakan {best_k} Cluster pada Slider Gradio nanti.")
69
70 # Jalankan Evaluasi
71 if not df_raw.empty:
72     evaluasi_cluster(df_raw, "Semua Tahun")
```



Sebelum model diterapkan, sel ketiga berfungsi sebagai tahap validasi statistik. Kode ini melakukan iterasi (looping) algoritma K-Means dengan variasi jumlah kluster (mulai dari 2 hingga 10). Program kemudian

menghasilkan dua visualisasi kunci: Grafik *Elbow Method* untuk melihat titik penurunan inersia optimal, dan Grafik *Silhouette Score* untuk mengukur seberapa rapi pemisahan antar kluster. Langkah ini membuktikan secara akademis bahwa pemilihan jumlah kluster (misalnya K=3) didasarkan pada bukti matematis, bukan asumsi semata.

4. Logika Inti Analisis (Aggregation & Modeling)

```
1 def logic_analisis(n_clusters, tahun):
2     # Cek Data
3     if df_raw.empty: return None, None, "Data Kosong"
4
5     # 1. Filter & Agregasi
6     if tahun != "Semua Tahun":
7         df = df_raw[df_raw['Tahun'] == int(tahun)]
8     else:
9         df = df_raw.copy()
10
11     df_agg = df.groupby('Provinsi').agg({
12         'Nama Peserta': 'count',
13         'Is_Juara': 'sum'
14     }).reset_index()
15
16     # Rename Kolom (PENTING)
17     df_agg.rename(columns={'Nama Peserta': 'Total_Peserta', 'Is_Juara': 'Total_Medali'}, inplace=True)
18     df_agg['Win_Rate'] = (df_agg['Total_Medali'] / df_agg['Total_Peserta']) * 100
19     df_agg = df_agg[df_agg['Total_Peserta'] > 0]
20
21     # Validasi Jumlah Data
22     if len(df_agg) <= n_clusters:
23         return None, None, f"Error: Data ({len(df_agg)}) kurang dari jumlah cluster ({n_clusters})."
24
25     # 2. Modeling
26     X = df_agg[['Total_Peserta', 'Win_Rate']]
27     scaler = StandardScaler()
28     X_scaled = scaler.fit_transform(X)
29
30     km = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
31     labels = km.fit_predict(X_scaled)
32     df_agg['Cluster'] = labels
33
34     # 3. Hitung Akurasi (Silhouette)
35     try:
36         score = silhouette_score(X_scaled, labels)
37         if score > 0.7: ket = "Sangat Baik"
38         elif score > 0.5: ket = "Baik"
39         elif score > 0.25: ket = "Cukup"
40         else: ket = "Buruk"
41         score_text = f"{score:.3f} ({ket})"
42     except:
43         score_text = "N/A"
44
45     # 4. Labeling & Formatting
46     # Urutkan label agar Tier 1 = Win Rate Terendah/Partisipan
47     centers = scaler.inverse_transform(km.cluster_centers_)
48     sorted_idx = np.argsort(centers[:, 1]) # Sort by Win Rate
49     mapping = {old: new for new, old in enumerate(sorted_idx)}
50     df_agg['Label'] = df_agg['Cluster'].map(mapping)
51
52     nama_tier = ['Perlu Evaluasi', 'Potensial', 'High Performance', 'Elite', 'Dominan']
53     df_agg['Kategori'] = df_agg['Label'].apply(lambda x: nama_tier[x] if x < len(nama_tier) else f"Tier ({x})")
54
55     # 5. Visualisasi
56     fig, ax = plt.subplots(figsize=(10, 6))
57     sns.scatterplot(data=df_agg, x='Total_Peserta', y='Win_Rate', hue='Kategori', palette='viridis', s=150, edgecolor='black', ax=ax)
58
59     # Label top 10
60     top = df_agg.nlargest(10, 'Win_Rate')
61     for _, r in top.iterrows():
62         ax.text(r['Total_Peserta'], r['Win_Rate'], r['Provinsi'], fontsize=9, fontweight='bold')
63
64     ax.set_title(f"Peta Sebaran ({tahun})")
65     ax.set_xlabel("Jumlah Peserta")
66     ax.set_ylabel("Win Rate (%)")
67     ax.grid(True, alpha=0.3)
68
69     # Label Output
70     df_out = df_agg[['Provinsi', 'Kategori', 'Win_Rate', 'Total_Peserta', 'Total_Medali']].sort_values('Win_Rate', ascending=False)
71     df_out['Win_Rate'] = df_out['Win_Rate'].round(2)
72
73     return fig, df_out, score_text
```

Sel keempat adalah otak sistem yang menggabungkan agregasi dan pemodelan. Pertama, data transaksional "Siswa" dikelompokkan (*groupby*)

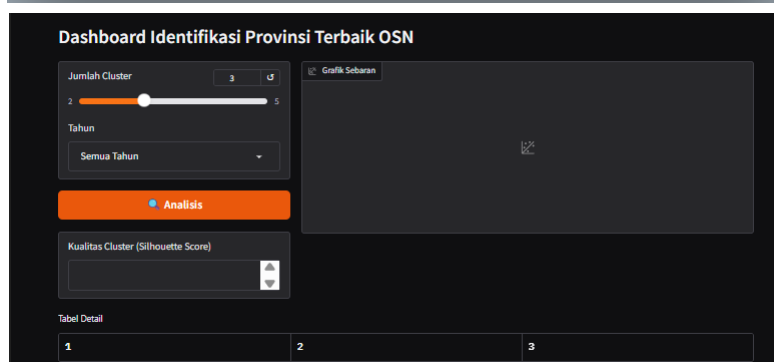
menjadi level "Provinsi". Di sini, variabel kunci `Win_Rate` diciptakan dengan membagi total skor prestasi dengan total jumlah peserta. Kedua, dilakukan normalisasi data (`StandardScaler`) agar variabel "Jumlah Peserta" (ratusan) tidak mendominasi "Win Rate" (persen). Terakhir, setelah klaster terbentuk, program melakukan pengurutan label (*sorting*) otomatis berdasarkan nilai rata-rata *Win Rate* tertinggi, memastikan label "Tier Elite" selalu disematkan pada kelompok dengan performa terbaik.

5. Implementasi Antarmuka (Deployment with Gradio)

```

1  # --- DEPLOYMENT GRADIO (FIXED NO THEME) ---
2
3  # Siapkan dropdown tahun
4  if not df_raw.empty:
5      years = sorted(df_raw['Tahun'].unique().tolist())
6      years_str = ["Semua Tahun"] + [str(y) for y in years]
7  else:
8      years_str = ["Semua Tahun"]
9
10 with gr.Blocks() as demo:
11     gr.Markdown("# Dashboard Identifikasi Provinsi Terbaik OSN")
12
13     with gr.Row():
14         with gr.Column(scale=1):
15             inp_k = gr.Slider(2, 5, value=3, step=1, label="Jumlah Cluster")
16             inp_thn = gr.Dropdown(years_str, value="Semua Tahun", label="Tahun")
17             btn = gr.Button("🔍 Analisis", variant="primary")
18
19             # Output Skor
20             out_skor = gr.Textbox(label="Kualitas Cluster (Silhouette Score)")
21
22         with gr.Column(scale=3):
23             out_plot = gr.Plot(label="Grafik Sebaran")
24
25             out_tbl = gr.Dataframe(label="Tabel Detail")
26
27             btn.click(logic_analisis, [inp_k, inp_thn], [out_plot, out_tbl, out_skor])
28
29     print(" Menjalankan Dashboard...")
30     demo.launch(share=True)

```



Tahap terakhir adalah pembangunan antarmuka pengguna (User Interface). Menggunakan kerangka kerja Gradio Blocks, kode ini menyusun

tata letak dashboard yang terdiri dari slider untuk pengaturan jumlah klaster dan dropdown untuk filter tahun. Interaksi input pengguna dan logika model dijembatani oleh event listener yang memicu perhitungan ulang secara real-time. Fitur kuncinya adalah parameter `share=True` pada fungsi `.launch()`, yang membuat tunnel aman dari server lokal ke internet publik, menghasilkan tautan URL (`.gradio.live`) yang dapat diakses dosen atau pengguna lain dari perangkat mereka.

BAB V

KESIMPULAN

Penelitian ini telah berhasil mengimplementasikan siklus CRISP-DM secara utuh untuk memecahkan masalah bias evaluasi pada kinerja provinsi di ajang Olimpiade Sains Nasional (OSN). Melalui penerapan teknik feature engineering berupa Win Rate dan sistem pembobotan prestasi (weighted scoring), kendala inkonsistensi data historis antara tahun 2009 hingga 2025 dapat teratasi dengan baik. Hasil pemodelan menggunakan algoritma K-Means Clustering terbukti efektif dalam memetakan provinsi ke dalam kelompok-kelompok yang memiliki karakteristik efisiensi unik, sehingga mampu memisahkan provinsi yang hanya mengandalkan kuantitas peserta dengan provinsi yang benar-benar memiliki kualitas pembinaan tinggi. Lebih lanjut, pengembangan dashboard interaktif berbasis web menggunakan Gradio telah berhasil menjembatani kesenjangan teknis, memungkinkan para pemangku kepentingan untuk memonitor dan menganalisis peta kekuatan pendidikan daerah secara objektif dan real-time tanpa memerlukan keahlian pemrograman.